Test Suite Minimization in LASSO Using Stimulus Response Matrices

Bachelor Thesis

submitted: 01.December 2025

by: Laith Philipp Sandouk
laith.sandouk@students.uni-mannheim.de
born May 15th, 2005
in Mannheim

Student ID Number: 1993811

Chair of Software Engineering
School of Business Informatics and Mathematics
University of Mannheim
D – 68159 Mannheim
Phone: +49 621-181-3912, Fax +49 621-181-3909
Internet: http://swt.informatik.uni-mannheim.de

Abstract

Modern software systems increasingly rely on automated test generation, resulting in test suites that may comprise thousands of cases and impose substantial costs in terms of execution time and computational resources [2]. This complexity heightens the need for *test-suite minimization*, which targets the elimination of redundant tests while preserving metrics such as fault-detection effectiveness and coverage [12, 9]. However, naive reduction strategies may discard tests crucial for revealing faults, thereby compromising software quality [12].

This thesis investigates test-suite minimization within the Large-Scale Software Observatorium (LASSO) [6], a research platform that applies language-independent *Stimulus–Response Matrices* (SRMs) to compactly encode the relationship between test cases, coverage requirements, and mutation analysis [13]. We present a comprehensive survey of established minimization techniques—including greedy heuristics [4, 11], exact optimization [7], search-based, and overlap-aware/meta-heuristics [1, 5]—and critically evaluate these methods according to coverage preservation, fault-detection retention, reduction ratio, computational efficiency, and SRM compatibility.

Drawing on empirical findings and industrial experience [10, 8, 1], this thesis proposes and integrates a minimization workflow tailored to LASSO and the SRM representation. The approach is designed to streamline regression testing and reduce resource costs for diverse programming environments, without sacrificing essential software quality indicators.

Contents

List of Figures

List of Tables

List of Abbreviations

CFG	Control Flow Graph
CI	Continuous Integration
GA	Genetic Algorithm
HGS	Harrold–Gupta–Soffa algorithm
ILP	Integer Linear Programming
LASSO	Large-Scale Software Observatorium
LSL	LASSO Scripting Language
OWL	Web Ontology Language
QIGA	Quantum-Inspired Genetic Algorithm
SRM	Stimulus–Response Matrix
TC	Telecommunications Company
TSM	Test Suite Minimisation

1. Introduction

Software testing is essential for ensuring that changes to a program do not introduce new faults. As systems grow in size and complexity, their test suites can become extremely large—automated generation tools such as Randoop and EvoSuite may produce thousands of test cases. While these suites achieve high code coverage and mutation scores, they often contain many redundant tests, leading to long execution times and increased maintenance costs. Test suite minimisation aims to remove redundant tests while preserving the effectiveness of the suite. This thesis investigates how to perform test suite minimisation directly on *Stimulus–Response Matrices* (SRMs) within the LASSO platform.

1.1. Background and Motivation

Regression testing verifies that software modifications do not break existing functionality. Software testing has become a major cost factor: early studies estimate that it can consume nearly half of a software system's development budget[3]. Large-scale test suites in modern projects may contain thousands of automatically generated test cases and take days or even weeks to execute[9]. This escalating complexity and cost demand effective strategies to keep test suites manageable. Test-suite minimisation offers a solution by removing redundant tests to reduce execution time and maintenance overhead. Yet naïve reduction risks omitting tests that detect faults and thereby undermines software quality[12]. Balancing reduction with coverage and fault-detection retention is therefore crucial.

1.2. LASSO Context

The Large-Scale Software Observatorium (LASSO) is a research platform developed at the University of Mannheim for large-scale software analysis. It integrates static and dynamic analysis, automated test generation and a domain-specific

language for orchestrating analysis pipelines. LASSO records test executions in *Stimulus–Response Matrices*. An SRM is a binary matrix where each row represents a test and each column represents a coverage requirement (statement, branch or mutant); an entry of 1 indicates that the test covers the requirement. Although LASSO provides powerful analysis capabilities, it currently lacks an integrated test-suite minimisation component. Adding minimisation would reduce redundant test execution and improve pipeline throughput.

1.3. Problem Statement

Let T be a set of test cases and let R be the set of coverage requirements. Each test $t \in T$ covers a subset $C(t) \subseteq R$. Executing the entire suite T yields full coverage $C(T) = \bigcup_{t \in T} C(t)$. The *test suite minimisation* problem asks for the smallest subset $S^* \subseteq T$ such that $C(S^*) = C(T)$. Solving this problem exactly is NP-complete because it reduces to the set-cover problem. Practical techniques therefore rely on heuristics and approximations that balance reduction ratio, coverage retention and computational cost.

1.4. Research Objectives and Questions

The goal of this thesis is to design and implement an SRM-based test-suite minimiser for LASSO. To achieve this goal, we address the following research questions:

1. Which test-suite minimisation techniques are most widely studied and effective according to the literature?
2. What criteria should be used to evaluate these techniques for integration into the SRM-based LASSO environment?
3. Which technique best balances reduction ratio, coverage retention and scalability for use with LASSO?

1.5. Implementation Strategy

Existing test-suite reduction tools (for example, the Java Software Reduction framework) often support only a single programming language or rely on language-specific program representations. Because LASSO analyses software written in diverse languages, this thesis adopts a language-independent strategy. Rather than integrating a pre-implemented program with limited language support, we propose to implement the minimisation algorithm ourselves directly on the SRM. This approach enables reduction of test suites generated for any language supported by LASSO and avoids dependencies on external tools with fixed language bindings.

1.6. Thesis Structure

The remainder of this thesis is organised as follows:

Chapter 2 introduces the fundamentals of code coverage, mutation testing, SRMs and formalises the test-suite minimisation problem.

Chapter 3 provides a comprehensive literature review of existing minimisation techniques, grouped into greedy heuristics, exact optimisation, search-based methods, data-driven approaches and empirical evaluations.

Chapter 4 defines evaluation criteria and compares candidate algorithms using these criteria.

Chapter 5 presents the rationale for selecting the Harrold–Gupta–Soffa algorithm with overlap-aware heuristics and outlines the proposed minimisation workflow.

Chapter 6 concludes the thesis and discusses future work.

2. Fundamentals

This chapter introduces the basic concepts underlying this thesis. It explains the metrics used to quantify the effectiveness of a test suite, describes the *Stimulus–Response Matrix* (SRM) representation used in the LASSO platform and formalises the test-suite minimisation problem.

2.1. Test Coverage and Mutation Score

Code coverage measures the proportion of program elements (statements, branches, etc.) exercised by a test suite. As additional test cases are generated automatically, coverage tends to plateau: after a certain point, adding more tests yields diminishing improvements. In this work we also consider *mutation testing*. Mutation testing introduces small syntactic changes (*mutants*) into the program and measures whether the test suite detects these faults. A *mutation score* is the ratio of mutants killed to the total number of mutants; a score of 1.0 means that all mutants are detected. Mutation analysis provides a stronger measure of test effectiveness than coverage alone because it evaluates the fault-detection capability of tests. We refer the reader to the mutation analysis chapter of the Fuzzing Book for a detailed explanation of mutation testing and its applications[13].

2.2. Stimulus–Response Matrices

The LASSO platform records test executions in *Stimulus–Response Matrices*. An SRM is a binary matrix $M \in \{0,1\}^{n \times m}$ where n is the number of test cases and m is the number of coverage requirements. Each row of M represents a test case and each column represents a requirement such as a statement, branch or mutant. An entry $M_{ij} = 1$ indicates that test i covers requirement j; otherwise $M_{ij} = 0$. SRMs enable coverage and mutation information to be stored compactly and manipulated algebraically. For example, summing each column yields the number

of tests covering each requirement, while summing each row yields the number of requirements covered by each test.

2.3. The Test-Suite Minimisation Problem

Let $T = \{t_1, \ldots, t_n\}$ be a set of test cases and let $R = \{r_1, \ldots, r_m\}$ be the set of coverage requirements. Each test $t \in T$ covers a subset $C(t) \subseteq R$ corresponding to the columns in the SRM with value 1. Executing the full suite T yields full coverage $C(T) = \bigcup_{t \in T} C(t)$. The *test-suite minimisation problem* asks for the smallest subset $S^* \subseteq T$ such that $C(S^*) = C(T)$. This problem is an instance of the classical *set-cover problem* and is NP-complete: no polynomial-time algorithm can guarantee an exact solution for arbitrary test suites. Practical approaches therefore employ heuristics or approximations. Existing techniques can be grouped into several broad categories:

- **Greedy heuristics.** These algorithms iteratively choose tests based on coverage information. Examples include the classical greedy algorithm that selects the test covering the most yet-uncovered requirements, and the Harrold–Gupta–Soffa (HGS) algorithm that prioritises tests covering rare requirements.
- **Exact optimisation.** The minimisation problem can be formulated as a weighted set-cover or integer linear programming (ILP) problem. ILP solvers can compute optimal solutions for moderately sized suites but do not scale well to very large matrices.
- **Search-based and meta-heuristics methods.** Genetic algorithms and quantum-inspired genetic algorithms treat minimisation as a search problem, evolving subsets of tests using a fitness function that balances size and coverage.
- **Data-driven techniques.** Approaches based on clustering or similarity metrics select representative tests by exploiting redundancy in the SRM. Overlap-aware heuristics consider the intersection between tests to avoid selecting tests with largely duplicated coverage.

These categories provide the framework for the subsequent literature review and evaluation of minimisation algorithms.

3. Literature Review

This chapter surveys the principal test-suite minimisation (TSM) approaches proposed in the literature. The techniques are grouped into greedy heuristics, exact optimisation, search-based methods, data-driven techniques and empirical evaluations.

3.1. Greedy Heuristics

3.1.1. Classical Greedy Algorithm

The simplest heuristic for set cover—and thus TSM—selects at each step the test case that covers the maximum number of yet-uncovered requirements. After selecting a test, all requirements it covers are removed from consideration and the process repeats until all requirements are covered. Although easy to implement, this algorithm does not consider overlap among tests and may select sub-optimal subsets. Despite its simplicity, it serves as a baseline and is widely used in practice.

3.1.2. Harrold–Gupta–Soffa Algorithm

Harrold, Gupta and Soffa proposed a variant that sorts requirements by the number of tests covering them[4]. The HGS algorithm first selects all tests that are the sole coverers of any requirement. It then repeatedly selects tests that cover requirements shared by exactly two tests, followed by those shared by three tests, and so on. In the case of ties, tests that appear most frequently in requirements of higher cardinality are chosen. This method emphasises rare requirements because such tests cannot be discarded without losing coverage. HGS typically produces smaller suites than classical greedy but can still miss the optimal solution. Smith et al. implemented HGS in a call-tree-based tool and found that reduced suites

contained 45 % fewer tests and consumed 82 % less execution time compared with the original suite[4].

3.1.3. Delayed Greedy (Concept Analysis)

Tallam and Gupta introduced a delayed greedy approach inspired by concept analysis[11]. Each test case is an object and each requirement is an attribute; coverage relationships form a context table. The algorithm removes rows or columns when one test's coverage implies another (for example, if test t covers all requirements of test u, then u is redundant). Only after these implications are resolved does it apply a greedy heuristic to select tests. Empirical results show that delayed greedy often produces smaller suites than HGS and classical greedy with comparable runtime. However, building and analysing concept lattices is computationally expensive for large SRMs.

3.2. Exact Optimisation Techniques

3.2.1. Integer Linear Programming and Weighted Set Cover

The TSM problem can be formulated as a weighted set-cover problem: each test case represents a set of instructions and has an associated cost (for example, execution time). The objective is to minimise total cost while covering all instructions. In weighted set cover, each set has a cost, and the goal is to minimise the cost of the selected sets. Exact solutions can be obtained by solving an integer linear program: minimise the sum of selected test costs subject to constraints that ensure every requirement is covered at least once. ILP solvers (for example, GLPK) use branch-and-bound and cutting planes to approximate or find optimal solutions, but the approach is computationally expensive and feasible only for moderately sized suites.

3.2.2. REDUNET: ILP with Network-Based Optimisation

Mongiovì et al. proposed REDUNET, which integrates set-cover ILP with a control-flow-graph (CFG) analysis[7]. The algorithm reduces the ILP problem size by pruning CFG nodes that are guaranteed to be executed even if excluded

from the optimisation step. By combining ILP and CFG analysis, REDUNET achieves optimal test-suite reduction while preserving code coverage. Experiments on ten projects showed reductions of up to 90 % and execution times faster than both pure ILP and HGS heuristics[7]. However, solving an ILP still incurs higher computational cost than greedy heuristics, and implementing CFG analysis requires access to internal program structure, which may not be available in LASSO's SRM.

3.3. Search-Based and Meta-Heuristic Methods

Search-based software engineering treats TSM as a search problem. Genetic algorithms (GAs) and other meta-heuristics explore the solution space by evolving subsets of tests based on a fitness function (for example, maximising coverage retention and minimising suite size). Hussein et al. summarise the history of these methods and note that TSM is NP-complete; they propose a quantum-inspired genetic algorithm (QIGA) that uses quantum superposition and rotation operators to improve convergence[5]. Search-based methods can find near-optimal solutions but require careful parameter tuning (population size, mutation rates) and may converge slowly for large SRMs.

3.4. Data-Driven Techniques and Clustering

Several works propose clustering or similarity-aware approaches to TSM. Bach et al. applied overlap-aware algorithms to prioritisation and selection in an industrial setting. They observed that considering the size of the overlap (intersection) between tests can lead to up to 50 % higher code coverage within a fixed time budget compared with traditional algorithms[1]. Their study also highlighted challenges such as high redundancy in coverage data and randomness due to flaky tests. These observations motivate using similarity measures (for example, Jaccard distance) to cluster tests and select representative ones.

3.5. Empirical Evaluations

Empirical studies reveal the trade-offs of TSM. Rothermel et al.'s early experiments (cited by subsequent work) showed that removing tests can significantly reduce execution time but may also degrade fault detection. Smith et al. developed a call-tree-based tool that implements HGS and several greedy variants for regression testing. Their experiments on Java programs found that the reduced suite contained 45 % fewer tests and consumed 82 % less execution time than the original suite; HGS first selects tests that uniquely cover a call-tree path and then iteratively chooses tests with maximal additional coverage[4]. Overlap-aware greedy heuristics recalculated cost effectiveness after each selection and achieved better trade-offs. Shi et al. analysed 1,478 failed builds from 32 open-source projects and introduced the *Failed-Build Detection Loss* (FBDL) metric: the percentage of failed builds where the reduced suite misses faults detected by the original suite. They found FBDL values up to 52.2 %, significantly higher than suggested by traditional TSM metrics[10]. Noemmer and Haas compared four minimisation algorithms on several open-source projects and reduced suites by over 70 %, but observed an average 12.5 % loss in fault detection capability[8]. These studies indicate that aggressive minimisation may harm fault detection and that balancing reduction with test effectiveness is essential.

4. Evaluation Criteria and Comparative Analysis

Selecting an appropriate minimisation algorithm for LASSO requires rigorous evaluation. This chapter defines criteria for evaluating candidate techniques and compares them qualitatively based on the literature reviewed in Chapter 3.

4.1. Evaluation Criteria

The following criteria are derived from the literature and tailored to LASSO's requirements:

Coverage Preservation The algorithm should retain code coverage and mutation score close to that of the original suite. Algorithms focusing on rare requirements (for example, HGS) or context reduction (for example, delayed greedy) tend to preserve coverage better than simple greedy.

Reduction Ratio The ratio of removed tests to the original suite. High reduction ratios (for example, more than 70 %) are desirable for efficiency but must not compromise effectiveness.

Fault Detection Retention The ability to detect real faults. Empirical studies show that minimisation can lead to missed faults (FBDL values up to 52.2 %)[10]; algorithms should minimise detection loss.

Computation Cost The algorithm must scale to large SRMs generated by LASSO. Greedy algorithms have low cost; ILP-based methods provide optimal solutions but can be slow. Search-based methods may require many iterations.

Use of SRM Data The algorithm should operate directly on SRMs without executing tests. Approaches that rely on program structure (for example, CFG analysis in REDUNET) may be impractical.

Implementation Complexity Simpler algorithms (classical greedy, HGS) are easier to implement and maintain in the LASSO codebase; complex search-based methods require parameter tuning and may be less predictable.

These criteria balance reduction, effectiveness and practicality.

4.2. Comparative Analysis of Candidate Algorithms

This section qualitatively assesses candidate minimisation algorithms against the criteria defined above. Table 4.1 summarises the comparison.

4.2.1. Classical Greedy

Coverage and Mutation. Classical greedy does not consider requirement rarity or overlap, which can lead to redundant selections and loss of coverage. Empirical studies show that simple greedy often produces larger minimised suites than other heuristics.

Reduction Ratio. Moderate; reduction depends on the distribution of coverage. May not achieve optimal reduction.

Fault Detection. Because it may select redundant tests, it tends to preserve fault detection better than aggressive pruning but may still remove tests with unique mutation coverage.

Cost. Very low; time complexity is roughly $\mathcal{O}(nm)$ for n tests and m requirements. Scales well to large SRMs.

Suitability for LASSO. Straightforward to implement on SRM. However, better alternatives exist.

4.2.2. Harrold–Gupta–Soffa Algorithm

Coverage and Mutation. By prioritising tests that cover rare requirements, HGS preserves coverage and mutation effectiveness.

Reduction Ratio. Achieves better reduction than classical greedy by avoiding redundant tests; suites have been reduced by 45–70 % in empirical studies.

Fault Detection. Good retention because unique requirements are preserved. Overlap-aware variants further improve effectiveness.

Cost. Slightly higher than classical greedy due to additional bookkeeping, but still efficient and scalable. Works directly on SRM.

Suitability for LASSO. A strong candidate. No need for program structure beyond SRM. Implementation complexity is modest.

4.2.3. Delayed Greedy (Concept Analysis)

Coverage and Mutation. By removing implied tests and requirements using concept lattice reductions, delayed greedy reduces context size and eliminates redundancy. It generally produces smaller suites than HGS.

Reduction Ratio. High; often achieves near-optimal reduction. However, context reduction may remove tests that provide unique mutant coverage if implication relations are computed solely on structural coverage.

Fault Detection. May risk greater fault detection loss if coverage relations do not reflect mutation information. Additional analysis is required to ensure mutation preservation.

Cost. Higher than HGS because building and analysing concept lattices is computationally expensive, especially for large SRMs.

Suitability for LASSO. Implementation complexity is higher. May be challenging to integrate concept-analysis libraries and maintain performance.

4.2.4. ILP-Based Optimisation / REDUNET

Coverage and Mutation. Can provide optimal reduction for coverage. REDUNET integrates CFG analysis and ILP to maintain code coverage and minimise execution time[7]. ILP can incorporate cost and weighting (for example, mutation score) directly.

Reduction Ratio. High; often achieving reductions of 90 % or more.

Fault Detection. Optimal coverage does not necessarily imply optimal fault detection. Without weighting mutants explicitly, ILP may remove tests that kill rare mutants.

Cost. Highest among candidates. Solving an ILP scales poorly; although REDUNET mitigates this via CFG analysis, it requires program structure and specialised solvers.

Suitability for LASSO. SRMs do not include CFG information. Adapting ILP requires mapping SRM entries to requirements and test costs; solving large ILPs may be infeasible within LASSO's time constraints.

4.2.5. Search-Based Methods

Coverage and Mutation. Genetic algorithms can optimise multiple objectives (size, coverage, mutation score). Quantum-inspired genetic algorithms further use quantum operators to improve convergence[5].

Reduction Ratio. Potentially high; search can find near-optimal solutions.

Fault Detection. Adjustable fitness functions allow weighting mutation score, but appropriate tuning is necessary.

Cost. Typically expensive due to iterative evaluations. Parameter tuning affects performance and solution quality. Not guaranteed to converge to optimal solutions.

Suitability for LASSO. Implementation complexity and computational cost make GA less attractive for routine SRM processing.

4.2.6. Overlap-Aware and Similarity-Based Techniques

Coverage and Mutation. By considering overlap between tests, these methods select tests that cover as many unique elements as possible and avoid those that mostly duplicate coverage[1]. Overlap-aware selection has been shown to achieve up to 50 % higher coverage within a time budget compared with traditional greedy approaches. Such heuristics often preserve fault detection because tests with unique coverage are preferred.

Reduction Ratio. Moderate to high; depends on the overlap distribution in the SRM.

Fault Detection. Overlap-aware methods often preserve fault detection because tests with unique coverage are preferred. However, high redundancy must be carefully handled to avoid dropping tests with unique mutation coverage.

Cost. Requires computing similarity measures (for example, Jaccard distance) between test rows; this is feasible for medium-sized SRMs but may be expensive for very large suites.

Suitability for LASSO. The SRM representation naturally supports computing overlaps and distances between test coverage rows. Implementation is manageable using matrix operations.

Summary

Overall, HGS augmented with overlap-aware heuristics strikes a favourable balance between reduction, coverage retention, computational efficiency and ease of implementation. Exact optimisation and search-based methods offer higher theoretical reduction but are unlikely to scale for routine use in LASSO. The next chapter therefore describes the rationale for selecting an HGS-based approach and outlines a workflow for its implementation on SRMs.

Table 4.1.: Qualitative comparison of selected minimisation algorithms

Algorithm	**Coverage**	**Reduction**	**Fault-detection**	**Cost**	**Complexity**
Classical greedy	medium	medium	high	very low	low
HGS	high	high	high	low	low
ILP/REDUNET	high	very high	medium	high	high
Overlap-aware	high	medium–high	high	medium	medium

5. Selected Approach and Workflow

Based on the comparative analysis in Chapter 4, this chapter motivates the choice of a Harrold–Gupta–Soffa (HGS) algorithm augmented with overlap-aware heuristics for use in LASSO. It also outlines a workflow for minimising test suites represented as stimulus–response matrices.

5.1. Rationale for Selection

The evaluation criteria highlight the need for a technique that balances reduction ratio, coverage preservation, fault detection retention, computational cost and ease of integration. Among the candidate algorithms, HGS excels in prioritising tests covering rare requirements, thereby preserving both coverage and mutation score. Empirical studies report reduction ratios between 45 and 70 % with minimal execution overhead. Incorporating overlap-aware heuristics addresses the limitation that HGS alone may select tests with substantial duplicated coverage. Overlap-aware selection considers the size of the intersection between tests and avoids redundant selections, achieving up to 50 % higher coverage within a fixed time budget[1]. These qualities make an HGS-based, overlap-aware approach the most suitable choice for integration into LASSO.

5.2. Minimisation Workflow

The proposed minimisation workflow comprises the following steps:

1. **Pre-processing.** Represent the SRM as a binary matrix where each row corresponds to a test case and each column corresponds to a coverage element (statement or mutant). Compute for each coverage element the number of tests that cover it. If available, record the execution time of each test as a weight.

2. **Identify unique coverage.** Select all tests that uniquely cover any element (that is, columns with coverage count of one). These tests form the initial reduced suite and mark the corresponding elements as covered. This ensures that no unique requirement is lost.
3. **Overlap-aware greedy selection.** While uncovered elements remain, iteratively select the test that maximises the ratio of newly covered elements to execution time or another hybrid cost metric. Recalculate cost effectiveness after each selection to account for overlaps. This step generalises the HGS strategy by emphasising both rarity and minimal overlap.
4. **Redundancy check.** After constructing a reduced suite, optionally perform a redundancy check: for each test in the reduced suite, determine whether its removal would leave some coverage element uncovered. Remove redundant tests to further reduce the suite.
5. **Mutation score verification.** Evaluate the mutation score of the reduced suite using existing mutation-analysis tools. If the score drops below an acceptable threshold, reconsider dropping tests that kill rare mutants.
6. **Output.** Produce an optimised SRM containing only the selected test cases and update the LASSO pipeline to use this suite for further analysis or execution.

5.3. Planned Evaluation

After implementation, the proposed approach will be evaluated on benchmark projects and real SRMs generated by LASSO. Key metrics will include:

- **Reduction Ratio:** Percentage of tests removed.
- **Coverage Retention:** Statement/branch coverage and mutation score of the reduced suite relative to the original.
- **Fault Detection Retention:** Ability to detect seeded faults or real defects.
- **Execution Time:** Time taken by the minimisation algorithm and the reduced suite.

The results will be compared with classical greedy, delayed greedy and ILP-based baselines to validate the effectiveness of the chosen approach. Depending on the findings, further enhancements—such as incorporating mutation scores into the cost metric or combining overlap awareness with clustering—may be explored.

6. Implementation Requirements

6.1. Header Text

All software (i.e. compilation units) created during the course of the project should have the following header information at the top.

DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

7. Conclusion

Test suite minimisation is essential for reducing redundancy and execution time in automatically generated test suites. A comprehensive literature review shows that while exact optimisation techniques and search-based methods can achieve high reduction, they incur substantial computational cost and require program structure or parameter tuning. Greedy heuristics, particularly the Harrold–Gupta–Soffa (HGS) algorithm, provide a balanced trade-off between reduction and preservation of coverage and mutation scores. Industrial studies highlight the benefits of considering overlap between tests to further improve selection effectiveness. This thesis therefore proposes to implement an overlap-aware HGS minimiser on Stimulus–Response Matrices in the LASSO platform. The forthcoming implementation will transform SRMs into optimised suites and empirically evaluate reduction ratio, coverage retention, mutation score and fault detection. The results will inform improvements and extensions, such as incorporating mutation scores into selection metrics or combining overlap awareness with clustering techniques.

Bibliography

[1] Thomas Bach, Artur Andrzejak, and Ralf Pannemans. „Coverage-Based Reduction of Test Execution Time: Lessons from a Very Large Industrial Project“. In: *2017 IEEE International Conference on Software Testing, Verification and Validation Workshops (ICSTW)*. IEEE. 2017, pp. 219–224.

[2] Barry W. Boehm. *Software Engineering Economics*. Englewood Cliffs, NJ: Prentice Hall, 1981.

[3] Barry W. Boehm. *Software Engineering Economics*. Englewood Cliffs, NJ: Prentice Hall, 1981.

[4] Mary Jean Harrold, Rajiv Gupta, and Mary Lou Soffa. „A Methodology for Controlling the Size of a Test Suite“. In: *ACM Transactions on Software Engineering and Methodology* 2.3 (1993), pp. 270–285.

[5] Hager Hussein, Ahmed Younes, and Walid Abdelmoez. „Quantum-Inspired Genetic Algorithm for Solving the Test Suite Minimization Problem“. In: *WSEAS Transactions on Computers* 19 (2020), pp. 143–153.

[6] Marcus Kessel. „LASSO – an Observatorium for the Dynamic Selection, Analysis and Comparison of Software“. Dissertation. University of Mannheim, 2022.

[7] Misael Mongiovì, Andrea Fornaia, and Emiliano Tramontana. „REDUNET: Reducing Test Suites by Integrating Set Cover and Network-Based Optimization“. In: *Applied Network Science* 5.86 (2020). DOI: 10.1007/s41109-020-00323-w.

[8] Raphael Noemmer and Roman Haas. „An Evaluation of Test Suite Minimization Techniques“. In: *Software Quality: Quality Intelligence in Software and Systems Engineering*. Springer, 2020, pp. 51–66.

[9] Gregg Rothermel and Mary Jean Harrold. „Empirical Studies of a Safe Regression Test Selection Technique“. In: *IEEE Transactions on Software Engineering* 24.6 (1998), pp. 401–419.

[10] August Shi et al. „Evaluating Test-Suite Reduction in Real Software Evolution“. In: *Proceedings of the 27th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA 2018)*. ACM. 2018, pp. 84–94.

[11] Sriraman Tallam and Neelam Gupta. „A Concept Analysis Inspired Greedy Algorithm for Test Suite Minimization“. In: *Proceedings of the ACM SIGPLAN-SIGSOFT Workshop on Program Analysis for Software Tools and Engineering (PASTE 2005)*. ACM. 2005, pp. 35–42.

[12] Shin Yoo and Mark Harman. „Regression Testing Minimization, Selection and Prioritization: A Survey“. In: *Software Testing, Verification and Reliability* 22.2 (2012), pp. 67–120.

[13] Andreas Zeller et al. *The Fuzzing Book – Mutation Analysis Chapter*. Accessed September 14, 2025. 2019. URL: `https://www.fuzzingbook.org/html/MutationAnalysis.html`.

Appendix

A.1. Example Appendix

This is a sample appendix entry.

Declaration of Authorship / Eidesstattliche Erklärung

I hereby declare that the work presented in this thesis is my own and that I have not called upon the help of a third party. In addition, I affirm that neither I nor anybody else has previously submitted this work or parts of it to obtain credits elsewhere. I have clearly marked and acknowledged all quotations and references that have been taken from the works of others. All secondary literature and other sources are marked and listed in the bibliography. The same applies to all charts, diagrams and illustrations as well as to all Internet resources. Moreover, I consent to my paper being electronically stored and sent anonymously in order to be checked for plagiarism. I know that if this declaration is not made, the paper may not be graded.

Hiermit versichere ich, dass diese Abschlussarbeit von mir persönlich verfasst ist und dass ich keinerlei fremde Hilfe in Anspruch genommen habe. Ebenso versichere ich, dass diese Arbeit oder Teile daraus weder von mir selbst noch von anderen als Leistungsnachweise andernorts eingereicht wurden. Wörtliche oder sinngemäße Übernahmen aus anderen Schriften und Veröffentlichungen in gedruckter oder elektronischer Form sind gekennzeichnet. Sämtliche Sekundärliteratur und sonstige Quellen sind nachgewiesen und in der Bibliographie aufgeführt. Das Gleiche gilt für graphische Darstellungen und Bilder sowie für alle Internet-Quellen.

Ich bin ferner damit einverstanden, dass meine Arbeit zum Zwecke eines Plagiatsabgleichs in elektronischer Form anonymisiert versendet und gespeichert werden kann. Mir ist bekannt, dass von der Korrektur der Arbeit abgesehen werden kann, wenn die Erklärung nicht erteilt wird.

Mannheim, September 15, 2025

Max Mustermann

Assignment of Usage Rights / Abtretungserklärung

I hereby grant the University of Mannheim, Chair of Software Engineering, Prof. Dr. Colin Atkinson, extensive, exclusive, unlimited and unrestricted usage rights to the work described in this thesis.

This includes the right to use the results in research and teaching. In particular, this includes the right to reproduce, distribute, translate, change and transfer the results to third parties, as well as any further results derived from them.

Whenever the results of my work are used in their original form or in a revised version, I will be mentioned by name as a co-author, in compliance with the copyright rules. This assignment does not include any commercial use.

Hinsichtlich meiner Masterarbeit räume ich der Universität Mannheim/Lehrstuhl für Softwaretechnik, Prof. Dr. Colin Atkinson, umfassende, ausschließliche unbefristete und unbeschränkte Nutzungsrechte an den entstandenen Arbeitsergebnissen ein.

Die Abtretung umfasst das Recht auf Nutzung der Arbeitsergebnisse in Forschung und Lehre, das Recht der Vervielfältigung, Verbreitung und Übersetzung sowie das Recht zur Bearbeitung und Änderung inklusive Nutzung der dabei entstehenden Ergebnisse, sowie das Recht zur Weiterübertragung auf Dritte.

Solange von mir erstellte Ergebnisse in der ursprünglichen oder in überarbeiteter Form verwendet werden, werde ich nach Maßgabe des Urheberrechts als Co-Autor namentlich genannt. Eine gewerbliche Nutzung ist von dieser Abtretung nicht mit umfasst.

Mannheim, September 15, 2025

Max Mustermann